

Kubernetes Cheat Sheet

Quick revision for DevOps, Kubernetes, kubectl and YAML

1. Kubernetes Basics

Concept	Meaning
Kubernetes	Open-source container orchestration platform
Cluster	Set of machines running Kubernetes workloads
Node	Worker machine that runs Pods
Control Plane	Manages cluster state and scheduling
Pod	Smallest deployable unit; runs one or more containers
Deployment	Manages replicated Pods and rolling updates
Service	Provides stable network access to Pods
Namespace	Logical isolation/grouping of resources

2. Common Kubernetes Workflow

Step	Typical action
1	Write Kubernetes YAML manifest
2	Create/update resources with kubectl apply
3	Scheduler assigns Pods to Nodes
4	Kubelet starts containers on the Node
5	Service exposes the application
6	Controller maintains desired replica count
7	Monitor with kubectl get/describe/logs
8	Roll out, scale or troubleshoot as required

3. Deployment YAML — Must Know

Concept	Meaning
apiVersion	API version, e.g. apps/v1
kind	Resource type, e.g. Deployment, Service, Pod
metadata	Name, labels and namespace
spec	Desired configuration of the resource
replicas	Number of desired Pod replicas
selector	Identifies Pods managed by a Deployment
template	Pod template used to create Pods
containers	Container image, ports, env and resources

4. Frequently Used kubectl Commands

Concept	Meaning
kubectl get pods	List Pods
kubectl get deployments	List Deployments
kubectl get services	List Services
kubectl get all	Show common resources
kubectl describe pod NAME	Detailed Pod information/events
kubectl logs POD	View container logs
kubectl exec -it POD -- sh	Open a shell in a container
kubectl apply -f file.yaml	Create/update resources from YAML
kubectl delete -f file.yaml	Delete resources from YAML
kubectl scale deployment APP --replicas=3	Change replica count

5. Kubernetes Objects & Networking

Concept	Meaning
Pod	Runs application containers
Deployment	Manages Pods, replicas and updates
ReplicaSet	Maintains the desired number of Pod replicas
Service	Stable virtual endpoint for Pods
ClusterIP	Service reachable inside the cluster
NodePort	Exposes a Service through a Node port
LoadBalancer	Uses cloud/load-balancer integration when supported
Ingress	HTTP/HTTPS routing into cluster Services
ConfigMap	Stores non-sensitive configuration
Secret	Stores sensitive configuration data

6. Namespaces, Contexts & Configuration

Concept	Meaning
kubectl get ns	List namespaces
kubectl create ns dev	Create a namespace
kubectl get pods -n dev	List Pods in a namespace
kubectl config get-contexts	List kubectl contexts
kubectl config current-context	Show active context
kubectl config use-context NAME	Switch cluster/context
kubectl get pods -o wide	Show Pod IP and Node
kubectl get pod POD -o yaml	Print resource YAML

7. Environment & Resource Settings

Concept	Meaning
env	Pass environment variables to containers
ConfigMap	Inject configuration without rebuilding image
Secret	Inject sensitive values
requests.cpu/memory	Minimum resources requested for scheduling
limits.cpu/memory	Maximum resources a container may use
livenessProbe	Checks whether the container is alive
readinessProbe	Checks whether the Pod is ready for traffic

8. Deployment & Rollout Commands

Concept	Meaning
kubectl rollout status deployment/APP	Watch rollout progress
kubectl rollout history deployment/APP	View rollout revisions
kubectl rollout undo deployment/APP	Rollback to previous revision
kubectl set image deployment/APP C=IMAGE	Update container image
kubectl scale deployment APP --replicas=N	Scale Deployment
kubectl rollout restart deployment/APP	Restart Pods managed by Deployment

9. Troubleshooting

Concept	Meaning
kubectl get pods	Check Pod status
kubectl describe pod POD	Inspect events and configuration

Concept	Meaning
kubectl logs POD	Check application logs
kubectl logs POD -c CONTAINER	Logs for a specific container
kubectl get events --sort-by=.lastTimestamp	Inspect recent cluster events
kubectl get endpoints SERVICE	Check Service endpoints
kubectl top pods	View Pod resource usage if metrics are available
kubectl top nodes	View Node resource usage if metrics are available

10. Docker + Kubernetes

Concept	Meaning
Docker	Builds/packages application containers
Kubernetes	Schedules, runs, scales and manages containers
Image	Immutable application package pulled by a container runtime
Deployment	Defines desired running application state
Service	Provides stable access to changing Pods
Registry	Stores container images, e.g. a private/public registry

11. Kubernetes YAML Example

Resource	Example
Deployment	<pre>apiVersion: apps/v1 kind: Deployment metadata: {name: myapp} spec: {replicas: 3, selector: {matchLabels: {app: myapp}}, template: {metadata: {labels: {app: myapp}}, spec: {containers: [{name: myapp, image: myapp:latest}]}}}</pre>
Service	<pre>apiVersion: v1 kind: Service metadata: {name: myapp-service} spec: {selector: {app: myapp}, ports: [{port: 80, targetPort: 8080}]}</pre>

12. Security — Must Remember

Concept	Meaning
RBAC	Controls who can perform which actions
ServiceAccount	Identity used by workloads running in the cluster
Secret	Use for sensitive values; avoid hard-coding credentials
NetworkPolicy	Controls allowed network traffic where supported
Least privilege	Grant only the permissions actually required
Image security	Use trusted images and scan them for vulnerabilities

13. Interview / Exam One-Liners

Concept	Meaning
Kubernetes	Container orchestration platform
Pod	Smallest deployable unit in Kubernetes
Node	Machine where Pods run
Control Plane	Manages the desired state of the cluster
Deployment	Manages replicated Pods and rolling updates
Service	Stable network endpoint for a set of Pods
Ingress	HTTP/HTTPS routing into cluster Services
ConfigMap	Non-sensitive application configuration
Secret	Sensitive configuration data
kubectl	Command-line tool for interacting with Kubernetes
Namespace	Logical isolation boundary for resources
Rolling Update	Gradually replaces old Pods with new ones

Quick revision rule: Understand the flow **Image** → **Pod** → **Deployment** → **Service** → **Ingress**, then practice **kubectl get** → **describe** → **logs** → **apply** → **rollout**.